

國立雲林科技大學資訊工程系

大學部實務專題

Department of Computer Science & Information
Engineering

National Yunlin University of Science & Technology

Senior Design

以 AI 為基礎動態預測飛機滑行碰撞預防系統

**Dynamic AI-based predictive flight taxiing collision
avoidance preventive in flight system**

邱忠憲、莊哲綸、詹閔勝

Chung-Hsien Chiu、Che-Lun Chang、Min-Sheng Chan

指導教授：張本杰 特聘教授

Advisor: Ben-Jye Chang, Ph.D.

中華民國 112 年 11 月

November 2023

國立雲林科技大學

資訊工程系

實務專題合格認可證明

專題學生：邱忠憲、莊哲綸、詹閔勝

實務專題題目：以 AI 為基礎動態預測飛機滑行碰撞
預防系統

**Dynamic AI-based predictive flight taxiing collision
avoidance preventive in flight system**

經評量合格，特此證明

指導教授簽名：_____

摘要

國際化的趨勢下，人們頻繁的往返各國，而其中最常被使用的交通工具便是飛機，飛機從滑翔機演變至今的噴射客機，於空中飛行已經有一套相當完整規範，畢竟稍有不慎發生空難，死傷無數也得耗費大量資源，所以飛機在空中時有強大的避障系統，不過地面上滑行時避障的能力遠不如空中，基本上僅僅只能依靠塔台人員指揮和機長自行目視，此時稍有不慎或是分心有可能還是會發生碰撞，地面上的碰撞雖然通常不太會造成大量人員傷亡，但是還是有可能造成飛機零件損壞，占用機場跑道之類的其他時間或金錢的成本，因此我們這此研究的這個系統便是輔助塔台和機長避免在機場滑行時發生碰撞。

本研究使用 **Hadoop** 建立分散式系統，再架設 NoSQL 資料庫 **HBase**，並利用天線接收飛機發送之 **ADS-B (Automatic Dependent Surveillance-Broadcast)** 訊號，此訊號包含飛機各項資訊如:GPS 位置、航班名稱等，將接收到的飛機存入系統的 **HBase**，利用 Hadoop 內建的 **MapReduce** 功能做資料的處理後再存回 **HBase**，**MapReduce** 可以幫助系統在處理大量資料時依然能有即時處理的能力，系統主程式將從 **HBase** 取出資料做運算，並判斷是否侵入對方的安全警示範圍，同時主程式也使用 **分離軸定理 (Separating Axis Theorem)** 偵測兩飛機碰撞的機率，並依據當前不同的警示等級，發送不同的警告內容，使管制人員能即時做出適當反應，也將飛機資料及當前狀況放上網頁監控。

此研究旨在降低和防止飛機於地面滑行時碰撞的機率，提前於即將發生碰撞前通知管制人員當下狀況，做出適當反應，如要求機長立即停止滑行，或是提前變換滑行道，避免不必要的資源浪費及人員傷亡。本研究為了解系統是否有效也還原了 2023/6/10 在日本羽田機場的碰撞事件，發現確實是能提前偵測並發送通知，以利管制人員做出反應，除了還原事件以外還有實際前往桃園國際機場，直接測試系統是否正常運作。

關鍵字：Hadoop, HBase, ADS-B, MapReduce, 分離軸定理(SAT)

致謝

這份畢業專題研究經過老師、學長的指導、建議下完成且完整呈現，首先，要感謝張本杰教授耐心的指導與寶貴的建議，教授對於事情的嚴謹、堅持、認真，引領本專題。不論在專題上的技術傳授還是需要用到的器材資源等，教授都給予滿滿的資源，也感謝教授在每次我們遇到問題及困難之處都給我們良好的解決辦法，我們在您的指導下都學到了許多寶貴的經驗和技能。接下來，要感謝實驗室的李尚昇學長、王子豪學長、林鼎軒學長、林鴻瑋，的指點以及許多建議，在我們遇到問題困難時，給予我們明確的方向讓我們獲得了更多的啟發。

最後，再次感謝您們的無私付出和細心指導。有了您們的幫助，我們才能夠完成本專題並獲得寶貴的學習經驗。這段時光將永存於我心中，成為我學術生涯中一段難忘的回憶。

目錄

摘要	-----	i
致謝	-----	ii
目錄	-----	iii
表目錄	-----	vi
圖目錄	-----	v
一、	緒論-----	1
二、	文獻探討-----	2
2.1	羽田事故原因-----	2
2.2	雲端系統與用戶端伺服器的差異-----	3
2.3	關聯式資料庫與非關聯式資料庫的差異-----	4
2.4	ADS-B -----	6
2.5	機場地面碰撞衝突風險-----	6
2.6	碰撞檢測演算法-----	8
2.7	GJK 演算法與 SAT 演算法的差異-----	9
三、	研究方法與內容-----	11
3.1	ADS-B 天線-----	11
3.2	雲端分散式系統-----	13
3.2-1	OpenStack-----	13
3.2-2	Hadoop-----	14
3.3	資料儲存-----	14
3.3-1	HBase-----	14
3.4	主要演算法-----	16
3.4-1	碰撞時間(Time to Collision ,TTC)-----	16
3.4-2	SAT(Separating Axis Theorem)+預測路徑-----	17
3.5	MapReduce-----	18
3.6	網頁即時顯示-----	20
3.7	發送警示訊息系統-----	22
四、	系統測試-----	23
4.1	情境一-----	23
4.2	機場實際測試-----	25
五、	結論-----	26
參考文獻	-----	28

表目錄

表 一	雲端及伺服器比較表-----	4
表 二	資料庫比較表-----	5
表 三	GJK 和 SAT 演算法比較表-----	10
表 四	有無系統輔助比較表-----	24
表 五	事故次數統計表-----	26
表 六	事故降低率-----	27

圖目錄

圖一	跑道停止線-----	2
圖二	A330-300 規格圖-----	2
圖三	羽田事故正面示意圖-----	3
圖四	羽田事故俯視示意圖-----	3
圖五	雲端系統圖-----	4
圖六	NoSQL 系統架構圖-----	5
圖七	ADS-B 訊號發射接收與地面接收示意圖-----	6
圖八	飛機前後滑行碰撞風險示意圖-----	7
圖九	飛機面向滑行碰撞風險示意圖-----	7
圖十	飛機於不同滑行道交會碰撞風險示意圖-----	7
圖十一	演算法-----	9
圖十二	SAT 判斷原理示意圖-----	10
圖十三	系統架構圖-----	11
圖十四	ADS-B 天線-----	12
圖十五	ADS-B 訊號傳輸流程-----	12
圖十六	ADS-B 天線接收到的資料對照圖-----	13
圖十七	OpenStack 系統實際圖-----	13
圖十八	Hadoop 系統圖-----	14
圖十九	HBase 展示圖-----	15
圖二十	HBase 系統圖-----	15
圖二十一	TTC 計算示意圖-----	16
圖二十二	TTC 範圍圖-----	16
圖二十三	A380 規格圖-----	17
圖二十四	SAT 預測路徑示意圖-----	17
圖二十五	主要演算發示意圖-----	18
圖二十六	MapReduce 運作圖-----	19
圖二十七	MapReduce 執行過程圖-----	19
圖二十八	MapReduce 應用示意圖-----	20
圖二十九	路徑變換顏色系統圖-----	20
圖三十	飛機高度圖示圖-----	21
圖三十一	飛機即時資訊表顯示圖-----	21
圖三十二	桃園機場系統實際測試圖-----	22

圖三十三	LINE 通知-----	22
圖三十四	羽田事件還原圖-----	23
圖三十五	增加系統後比較圖-1-----	23
圖三十六	警告訊息發送圖-----	24
圖三十七	增加系統後比較圖-2-----	24
圖三十八	桃園機場實測圖-----	25
圖三十九	機場飛機實拍圖-----	25
圖四十	系統狀態改變圖-----	26

一、 緒論

人類在現今 21 世紀的航空技術上相當成熟，透過飛機往返各國可以說既便利又省時，航空的安全規範避障能力也可以說相當完善，但是這通常僅限於天空中飛行的狀態，畢竟飛機一旦於空中失事存活率將非常的低，所以空中的避障系統肯定比較完善。不過飛機在地面就不會有事故嗎?答案是絕對會有，只是跟空中失事相比絕對輕微很多，大部分都是財損和時間的損失居多，比如 2023 年 6 月 10 日日本羽田國際機場長榮航空和泰國航空於跑道滑行時發生擦撞事故，此事故使得羽田機場部分跑道暫時封閉，多架班機時間受到延誤，所以飛機於地面上還是有發生事故的風險，因為現行的機場調度與管制由塔台人員做管理與調度，再加上機長自行目視和經驗判斷，此時若機長經驗不足或是判斷錯誤，塔台管制人員也沒注意到時就容易有擦撞事故的發生。

現行機場滑行的調度與管制是透過人工的方式做處理，假設經驗不足或是稍微恍神事故發生的風險就會大大的增加，因此本專題想透過開發軟體來輔助管制人員或是機長，這樣可以提升機場效率，減少事故發生的風險，更可以減少事故發生後需要封閉機場部分區域以及飛機延誤所帶來的維修、旅客賠償金和時間的成本，讓機場多一份保障。

本專題透過接收 ADS-B 訊號獲取飛機資訊，使用分散式系統以及 NoSQL 提高系統的資料吞吐量、系統穩定性和高擴充性，藉由演算法來計算飛機若持續以當前速度、方向前進是否會有碰撞風險，風險高則發送訊息通知管制人員，並且也有監控網頁，會即時顯示出當前飛機的狀況，同時也能顯示飛機資訊。

本專題期望能夠透過軟體運算在飛機真正發生擦撞的前 40~50 秒提前發送警訊，網頁上飛機狀態也會做改變，藉此提醒飛機可能太近了，或是若持續前進可能會發生碰撞，讓管制人員和機長來得及做出適當反應，如減速、加速、改道等等，降低地面事故發生，保護生命安全也減少財損和時間的損失。

二、 文獻探討

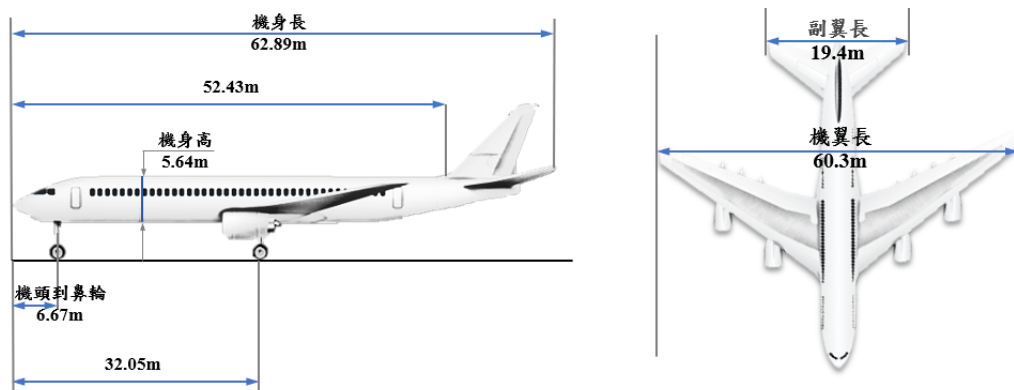
2.1 羽田事故原因

2023/6/10 羽田機場 L14 滑行道發生泰航碰撞長榮的事件，此次事件由多個人為的失誤造成，日本的 A.I.P(Aeronautical Information Publication)有規定若要等待進入跑道，飛機需在停止線後等待，且機身任何部件皆不得越線圖一為停止線，若越線則視為入侵跑道，立即停飛。



圖一、跑道停止線

A330-300



圖二、A330-300 規格圖

長榮航空的 BR189 當時停止於 L14 滑行道上等待起飛，泰國航空 TG683 由 L 滑行道行經 BR189 欲前往 L15 滑行道等待起飛，圖二為 A330-300 的規格圖，TG683 和 BR189 皆為 A330-300 型客機，TG683 在經過 BR189 時機長誤判飛機機翼寬可以通過導致擦撞 BR189 的水平尾翼，另一個失誤則是 BR189 為了避免飛機機鼻壓線因此停靠位置過度後面，此次事故造成 L 滑行道關閉長達兩小時。



圖三、羽田事故正面示意圖



圖四、羽田事故俯視示意圖

2.2 雲端系統與用戶端伺服器的差異

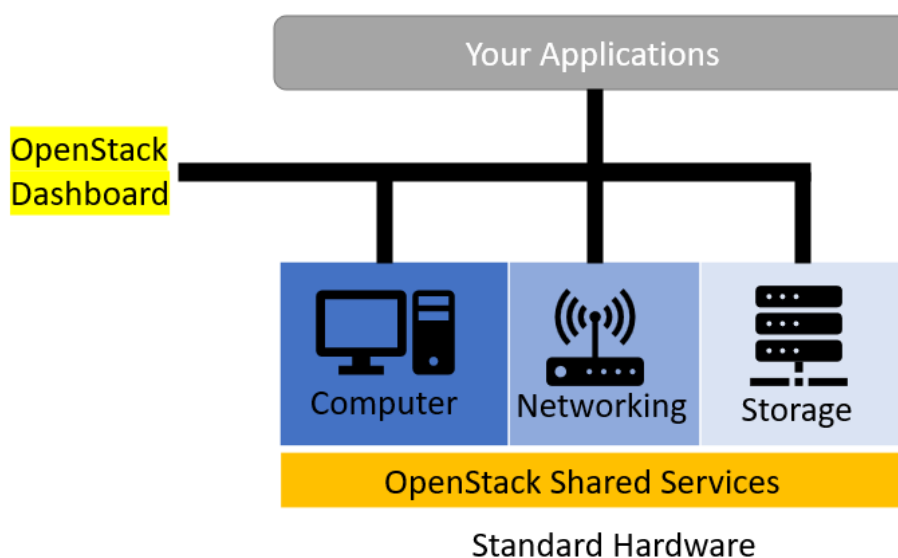
雲端系統是基於分散式架構相較於用戶端伺服器，其資源的存取和處理均分布在多個伺服器上，解決了用戶端伺服器無法改善的問題如:無法延展、硬體成本高、難以擴充等等缺點。並且雲端系統為了有效地做系統資源的分配，所以採用大量的且小型的虛擬主機來取代高成本的運算主機。在網際網路中都是用戶端和伺服器達成連線為基礎結構，再由用戶端像伺

伺服器提出請求伺服器才會做出相應的回應。而雲端伺服器不但會做出回應，還會替用戶執行程式並且將資料存取。

本次研究會使用 OpenStack 這套軟體，因為 OpenStack 是一套以基礎設施即服務(Infrastructure as a Service, 簡稱 IaaS)為主的雲端系統平台，而且 OpenStack 是由 Rackspace 和 NASA 共同開發的開放原始碼雲端 IaaS 層管理平台，加上 OpenStack 已經有雲端架構中所有的重要部件，像是運算點，網路點，存放點。所以我們會以此軟體為主，進行延伸。

特點	雲端系統	用戶端伺服器
資源位置	遠端的雲端伺服器	分散在用戶或本地伺服器
存取速度	取決網路品質	本地存取較快，不受網路影響
擴展性	高，可動態配置	低，硬體和成本考量
儲存空間	根據需求動態配置	須預先考量配置
便利性	高	低

表一、雲端及伺服器比較表



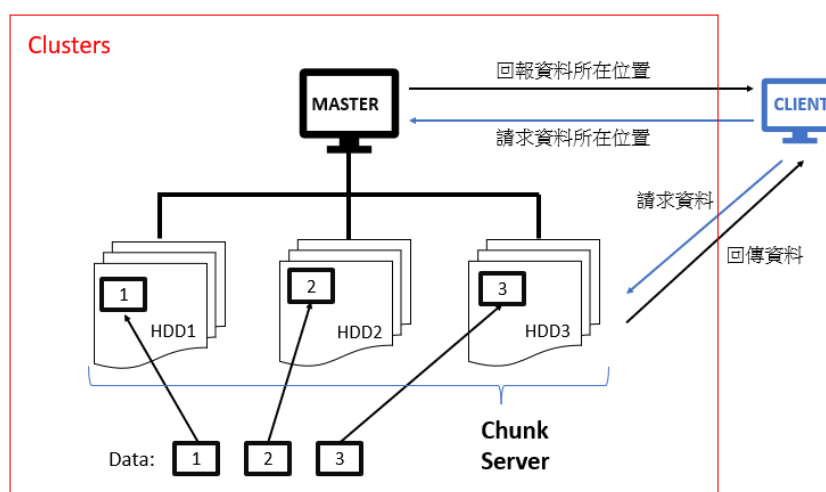
圖五、雲端系統圖

2.3 關聯式資料庫與非關聯式資料庫的差異

非關聯式資料庫的數據模型支援多種形態，如文檔、鍵值對、列族等等，而關聯式資料庫採用的是結構化的表格形式，在架構上非關聯式資料庫具有動態調整的彈性，比起關聯式資料庫使用的預先定義的固定結構，更能

適應非結構化或試辦結構化的數據。在擴展性上非關聯式資料庫不像關聯式資料庫透過垂直擴展增加伺服器的計算與記憶體資源，而是採取水平擴展，更適合處理大規模數據。而在現實實務上，關聯式資料庫確保了數據的原子性、一致性、隔離性、持久性等等，有較嚴謹的資料讀取寫入的規定，非關聯式資料庫則是強調在分散環境下的最終一致性，會因為閒時關係，針對目的而採取相對應的調整。

本次研究採用 HBase 作為我們的資料庫，HBase 是建構於 Apache Hadoop 的 HDFS(Hadoop Distributed File System)之上，有著 Hadoop 的分散式文件儲存和分散式運算的優勢。HBase 作為一個分散式系統，擴展性是最大的優勢，可以輕鬆應對數據量的增加，而無須中斷服務，而且 HBase 是為了高效的隨機讀寫而設計，可以讓用戶動態的調整數據模型，不會受到先前定義的模式限制。且基於分散式架構，Hbase 能夠實現高可用性和容錯性，即使有節點失敗，也不會讓整個系統崩壞。



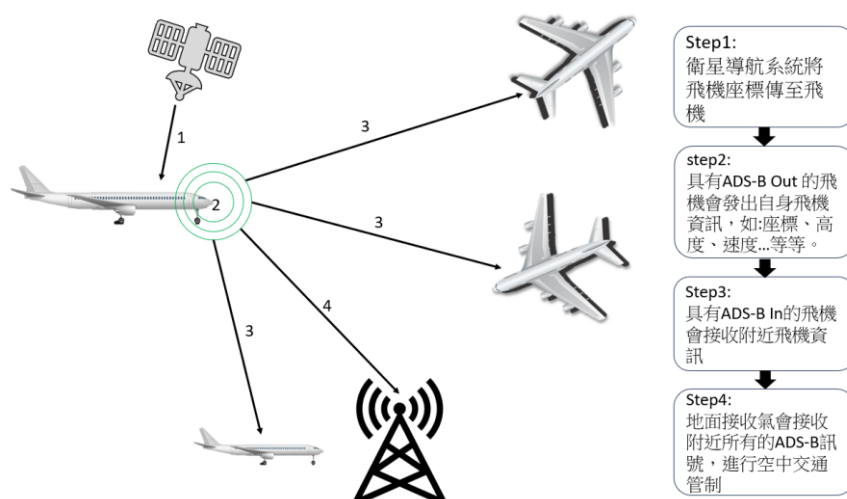
圖六、No SQL 系統架構圖

特點	關聯式資料庫	非關聯式資料庫
資料結構	結構式	非結構式、半結構式
彈性	低，須預先定義表格	高，可儲存不同結構資料
效能	差，難處理複雜查詢	高，可處理大量資料
擴展性	低，擴展成本高	高，可以隨時動態調整
適用場景	需要嚴格控管的資料	需要高擴展性與高速讀取的資料
資料一致性	透過內外約束確保資料一致	強調資料的最終一致性

表二、資料庫比較表

2.4 ADS-B

ADS-B(Automatic Dependent Surveillance-Broadcast) 是一種自動航空電子技術，其中包含了「ADS-B In」和「ADS-B Out」兩種服務。ADS-B Out 通過飛機上搭載的週期性發射器廣播每架飛機的資訊，作為當前空中交通管制員當前飛機位置的依據，比現有基於雷達的資訊更精準。ADS-B In 則是提供飛機接收來自 FIS-B、TIS-B 以及其他 ADS-B 的資訊，用來接收附近飛機的直接通信，並且透過 FIS-B 和 TIS-B 應用程式提供交通和官方生成的圖形天氣資訊。ADS-B 主要依賴高完整度的 GPS 導航源與一個數據鏈(ADS-B 單元)，最常使用的就是 109MHz 的數據鏈，其本質上是經過修改的 S 模式應答機。其中包含了飛機的位置、速度、高度…等等資訊，並且會每秒向地面管制站和周遭配備 ADS-B In 的飛機提供數據，使得飛機被看見並提高安全性，ADS-B 數據可以記錄和下載，用於飛行後的數據分析、航班追蹤、計畫和調度的數據基礎設施。

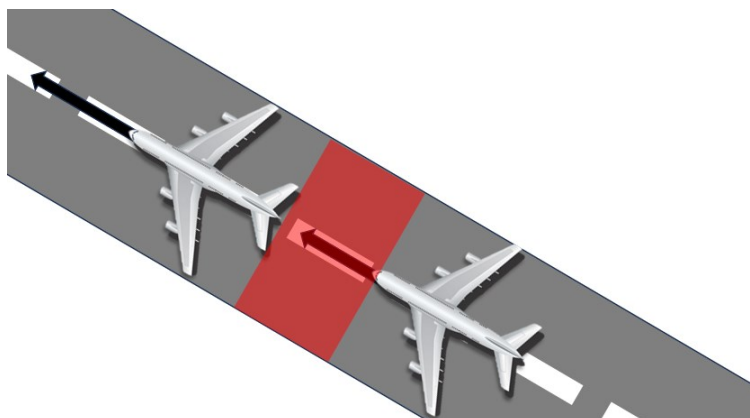


圖七、ADS-B 訊號發射接收與地面接收示意圖

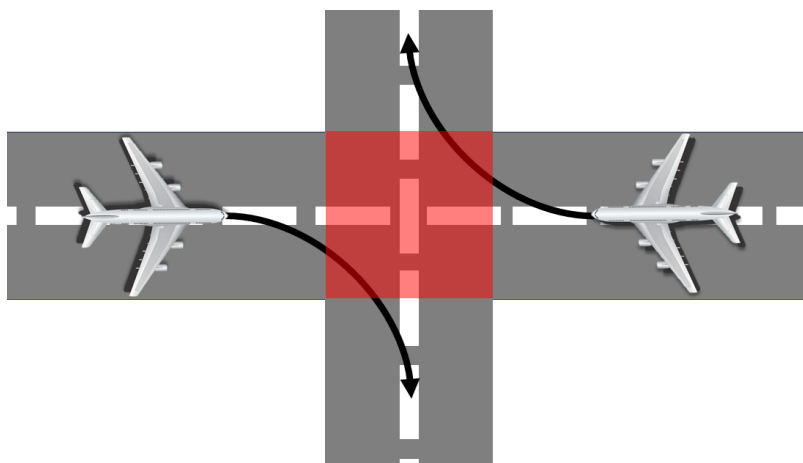
2.5 機場地面碰撞衝突風險

在機場滑行道上若有數台飛機同時進行滑行就會有碰撞的風險。目前的機場都是以地面管制人員為主要判斷方式。若以飛機上的人員進行判斷，容易因為視野受限而判斷錯誤，飛機在滑行道的碰撞分為兩架飛機於前後滑行，有可能面臨後方飛機滑行過快碰撞前方飛機的風險，另外在滑行道上若有兩台飛機以不同方向前進，將會面臨相撞的風險，在相交的兩條滑行道上若有兩台飛機同時前往交會處，也會面臨兩架飛機擦撞的風險，雖然在各國的

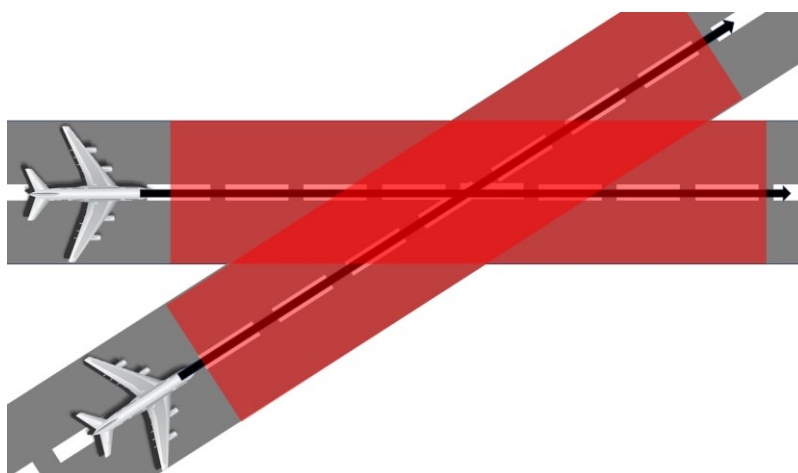
機場安全守則都有明確且相關的規定，但由於飛機的體積太過於龐大，並且駕駛的視野受限，所以仍然會有許多碰撞的事故發生。



圖八、飛機前後滑行碰撞風險示意圖



圖九、飛機面向滑行碰撞風險示意圖



圖十、飛機於不同滑行道交會碰撞風險示意圖

2.6 碰撞檢測演算法

本次專題採用 TTC(Time to collision)、兩圓切線是否相交和分離軸定理(SAT)作為碰撞檢測的主要方法。TTC(Time to collision)是一種基於動態資訊的檢測方法，通常用於預測未來某個時刻兩物體是否會發生碰撞，TTC 可以使用不同的數學模型如:速度、加速度等等的動態參數，提供詳細的碰撞風險評估，其基本構思是透過計算兩物體之間的相對速度和距離，來估計他們相遇所需要的時間。透過 TTC 計算過後我們利用圓形的特性，以飛機座標畫出三個不同距離的圓形警戒範圍，再使用兩圓形切線的特性去計算兩個圓形是否有相交的風險，若兩架飛機進入警戒範圍，使用分離軸定理更準確預測兩架飛機碰撞的風險。

Algorithm TTC

```
1: Input: airplanes information form ADS-B
2: Output: warning message
3: Initialization Warning G(the speed of aircraft on the
   ground 50km/hr(13.8m/s)*primary warning time plus
   reaction time)/2=417m,
4: Y(the speed of aircraft on the ground
   50km/hr(13.8m/s)*secondary warning time plus reaction
   time)/2=250m,
5: R(the speed of aircraft on the ground
   50km/hr(13.8m/s)*final warning time plus reaction
   time)/2=138m
6: While airplane1, airplane2
7:   Calculate the distance between two planes
8:   if distance between two planes <= G
9:     return G;
10:  else if distance between two planes <=Y
11:    send warning message to phone
12:    return Y;
13:  else if distance between two planes <=R
14:    SAT(airplane1,airplane2);
15:    send warning message to phone
16:    tell driver stop or turn left or turn right
17:    return R;
```

Algorithm SAT	
1: Input:	<i>Airplane1 a1, a1.Lat, a1.Lon , a1.heading, a1.speed, Airplane2 a2, a2.Lat, a2.Lon , a2.heading, a2.speed</i>
2: Output:	<i>collision or not</i>
3:	<i>Initialization T=20sec, wingspan=79.75m,length=72.73m</i>
4:	<i>If warning = R</i>
5:	<i>a1.location(a1.Lon±wingspan/2,a1.Lat-length/2) * a1.heading</i>
6:	<i>a1.next(a1.Lon±wingspan/2,a1.Lat + length/2) * T * a1.heading</i>
7:	<i>a2.location (a2.Lon ± $\frac{wingspan}{2}$,a2.Lat - $\frac{lenght}{2}$) * T * a2.heading</i>
8:	<i>a2.next (a2.Lon ± $\frac{wingspan}{2}$,a2.Lat + $\frac{lenght}{2}$) * T * a2.heading</i>
9:	<i>find a1,a2 location Lon and next Lon max , min value</i>
10:	<i>if a2.max or a2.min between a1.max and a1.min</i>
11:	<i>return they will collision</i>
12:	<i>else if a1.max or a1.min between a2.max and a2.min</i>
13:	<i>return they will collision</i>
14:	<i>else if a1.max or a1.min not between a2.max and a2.min</i>
15:	<i>return they are not collision</i>
16:	<i>else if a2.max or a2.min not between a2.max and a2.min</i>
17:	<i>return they are not collision</i>

圖十一、演算法

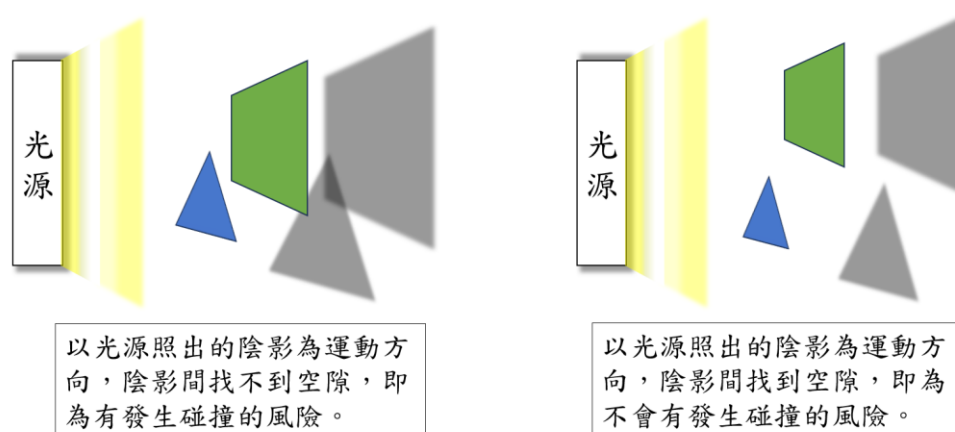
2.7 GJK 演算法與 SAT 演算法的差異

GJK 演算法(Gilbert-Johnson-Keerthi)採用計算凸多邊形上特定點之間的向量來判斷是否相交和 SAT 演算法(Separating Axis Theorem)又稱分離軸定理採用投影和軸的概念來進行碰撞檢測，兩者雖然利用不同的數學方式，但皆是廣泛應用在碰撞檢測的方法，其各自有獨特的優點和應用場景。

GJK 主要應用在判斷兩個凸集是否相交，基本原理為通過迭代運動的特定方向，擴展凸多邊形的範圍，直到找到兩圖形相交的點或是兩個圖形不相

交，GJK 演算法的優勢在於適用於各種形狀的物體，尤其是凸多邊形的處理效果尤佳，但是 GJK 演算法的計算過程較為複雜，必須經過多次的調整與優化才能達到最佳效果。

SAT 演算法又稱作分離軸定理，是一種簡單且直觀的物體碰撞檢測方法，其基本原理為透過將物體投影到不同的軸上，檢查是否重疊，如果兩個物體在任何一個軸上的投影皆不重疊，則兩物體不相交，SAT 演算法的優勢為較為容易理解與實現，並且可以同時檢測多個物體，但是在處理凸多邊形的時候效能會稍微差一點。

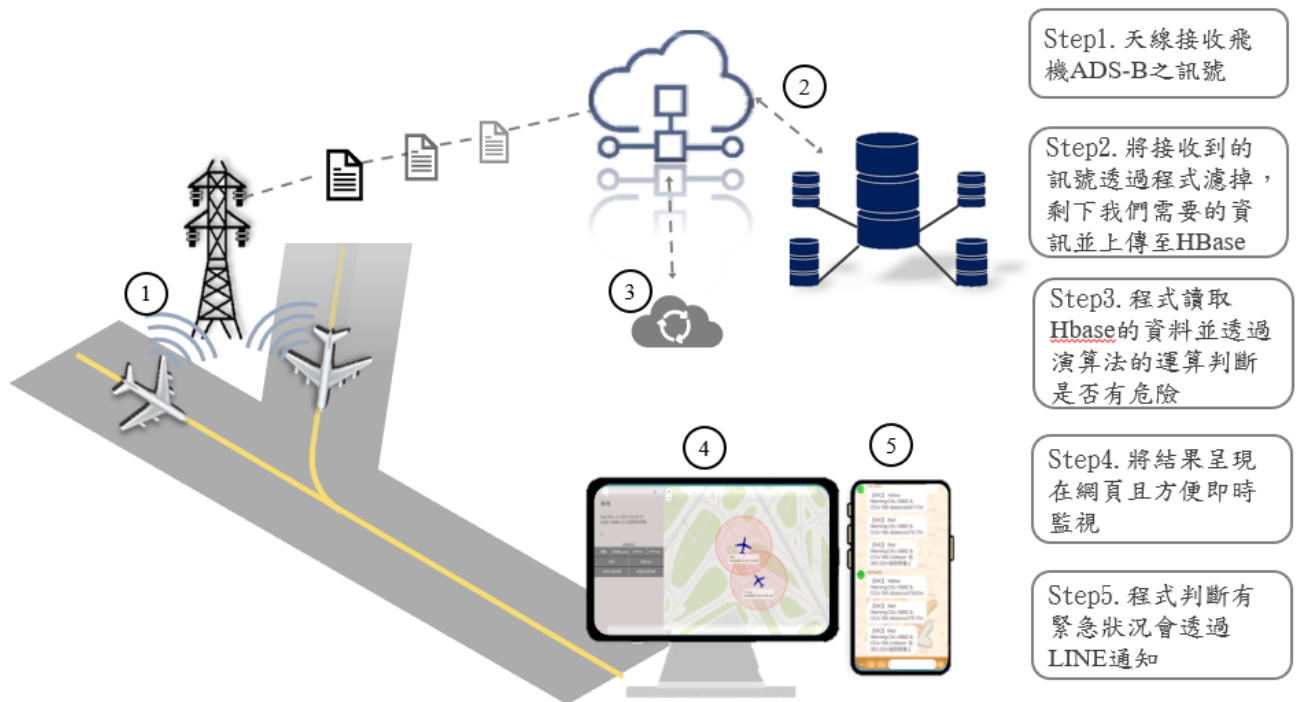


圖十二、SAT 判斷原理示意圖

特點	GJK 演算法	SAT 演算法
適用形狀	凸集與複雜形狀	任意形狀的物體
實現難度	複雜，需要調整與優化	簡單，容易理解
效能	處理凸多邊形效能較佳，但凹多邊形與曲面時較差	對於凸多邊形效能較差，但對於任意凸型的通用性高
適用場景	需要考慮複雜形狀和向量的場景	對效能和直覺性較地場景
基本原理	計算凸多邊形上特定點的向量判斷	透過軸與投影的概念判斷

表三、GJK 和 SAT 演算法比較表

三、 研究方法與內容



圖十三、系統架構圖

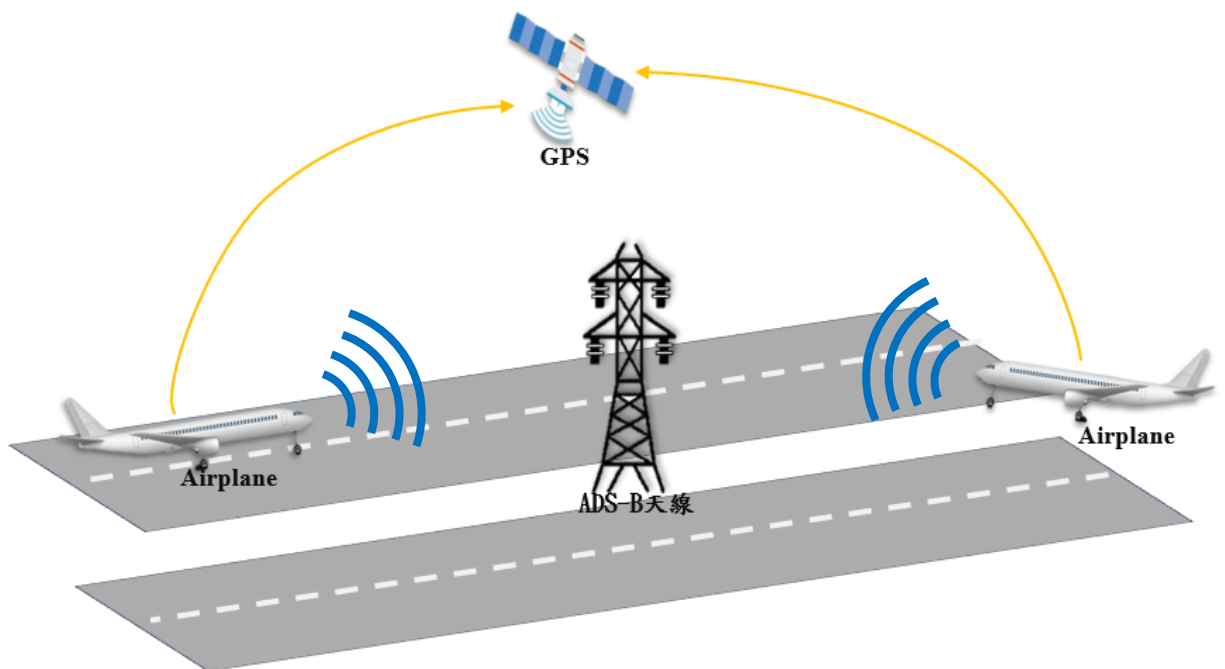
3.1 ADSB 天線

由於本專題需要飛機的相關資訊以及條件，不論是在機場跑道上滑行(Taxing)或是即將起飛(take off)和剛降落(landing)的飛機，而可以提供該資訊的即是 ADS-B 的訊號，因此本專題需要一組能持續且穩定的接收到該訊號的天線。

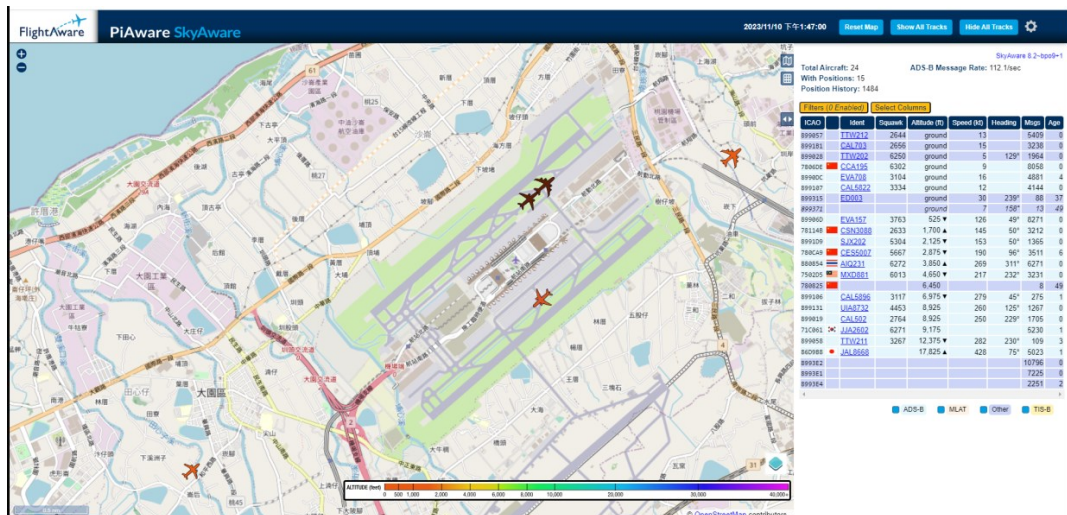
ADS-B(Automatic Dependent Surveillance-Broadcast)是一種飛機監視技術，由地面站和機載站構成，以網狀、多點對多點方式完成數據雙向通信。主要能實施空對空、地對空監視，使用於輔助航機以及地面航管系統，提高航機在飛行中的安全性，飛機會通過衛星導航系統(GPS)確定其所在位置，並定期進行廣播，使飛機可被航管人員追蹤，飛機會自動廣播飛機本身的資訊(ex:航高、航角、Gps、速度等)，系統廣播頻率在 1090MHZ，透過具有 RTL2832U +R820T 這兩個晶片的天線接收後，即可對 ADS-B 訊號做解碼，獲得該飛機的資訊，再輸出 json 檔將資料上傳雲端上的 HBASE 資料庫。



圖十四、ADS-B 天線



圖十五、ADS-B 訊號傳輸流程



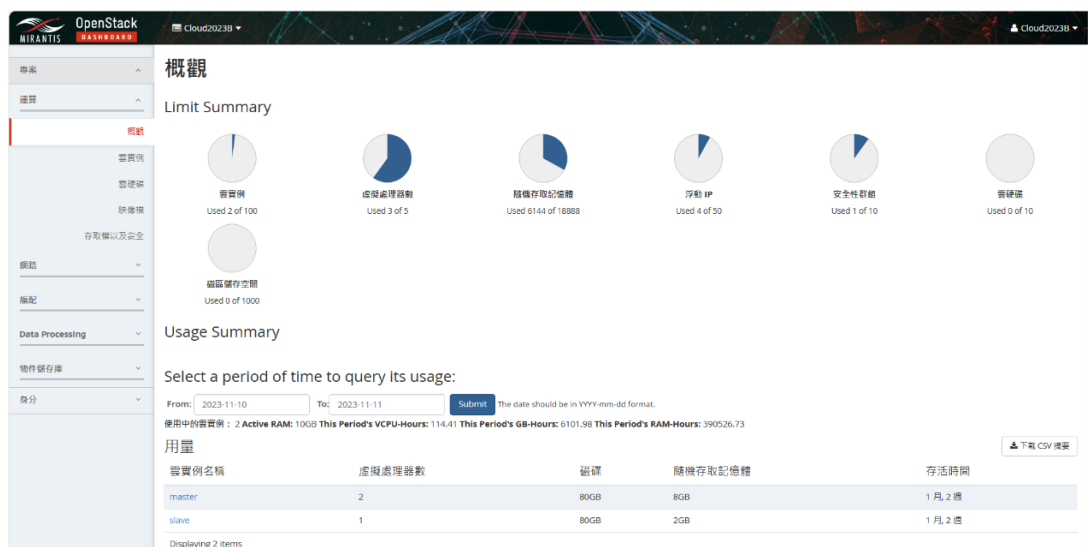
圖十六、ADS-B 天線接收到的資料對照圖

3.2 雲端分散式系統

3.2-1 OpenStack

專題上需要建立分散式系統，因此在 OpenStack 上建立多台虛擬機做使用，OpenStack 是一個開源雲計算平台，提供可擴展性的平臺即服務(PaaS)基礎設施及服務(IaaS)，可部署公用雲和私有雲中。

因為專題需要利運雲計算提高運算效率，同時利用其高擴展性的特點使專題能隨時擴展所需資源，不受限於個人電腦的運算能力和效能，更有效幫助系統開發。

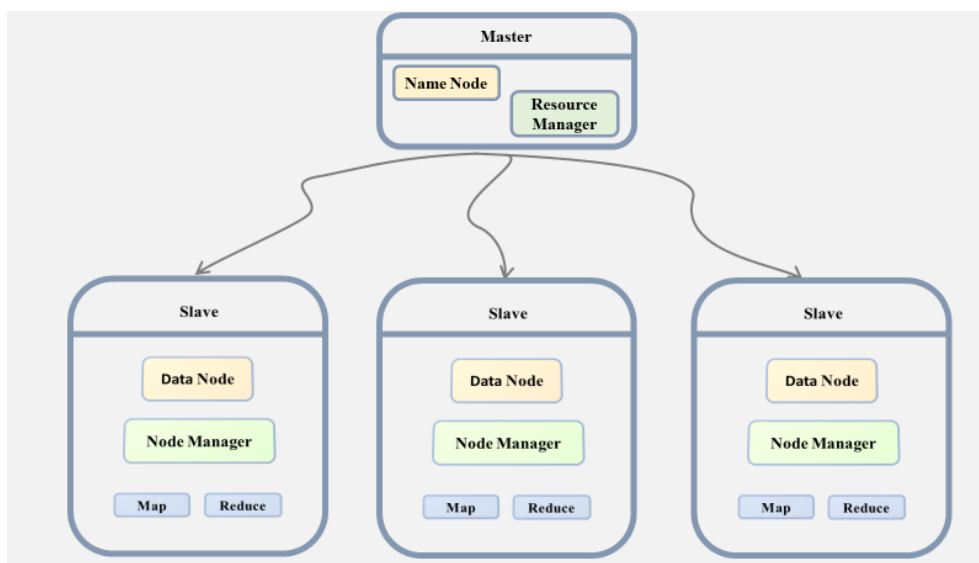


圖十七、OpenStack 系統實際圖

3.2-2 Hadoop

專題採用的分散式系統為 Hadoop，Hadoop 能透過水平擴展的方式將幾台效能普通的電腦做資源管理，進而提升其運算資源與效能，並且相較不斷擴充單一電腦的資源所產生的成本，Hadoop 能以較節省成本的方式達成效能的提升。

Hadoop 有兩大架構，分別是處理大量數據存取的 HDFS(Hadoop Distributed File System)和大量數據運算的 YARN(Yet Another Resource Negotiator)，這兩大架構本專題都會使用到，因此選用 Hadoop 作為分散式系統。



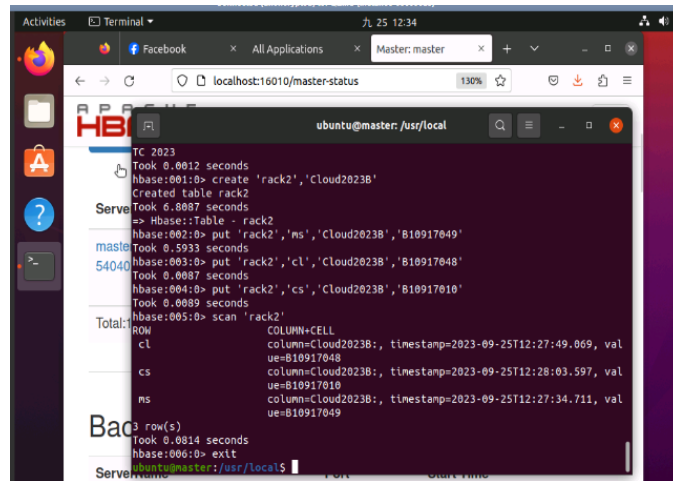
圖十八、Hadoop 系統圖

3.3 資料儲存

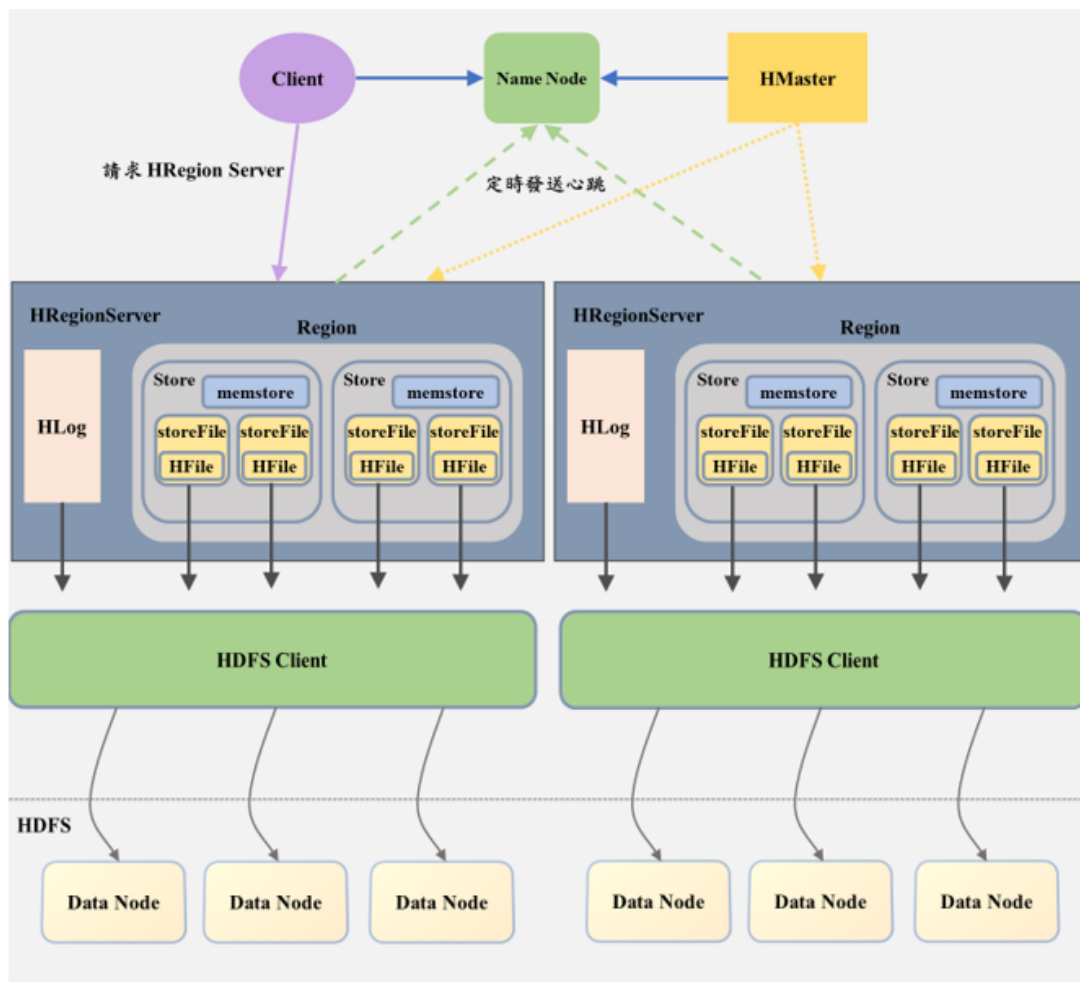
3.3-1 HBase

本專題需要具備能**同時存取和讀取**大量的資料的資料庫，**HBase**正具備這些功能，HBase 為 Hadoop 的擴展應用屬於 NoSQL，HBase 使用到 Hadoop 的 HDFS 系統。

HDFS 可以以低成本換來大量的儲存空間，加上 HBase 能夠處理大規模數據，具備高容錯能力、高擴展性和高成本效益，本專題從 ADS-B 天線抓取飛機資訊存入資料庫，到程式的運算都需要頻繁的讀寫，且需要低延遲，HBase 正好能達成，對於本專題是非常合適的資料庫。



圖十九、Hbase 展示圖



圖二十、HBase 系統圖

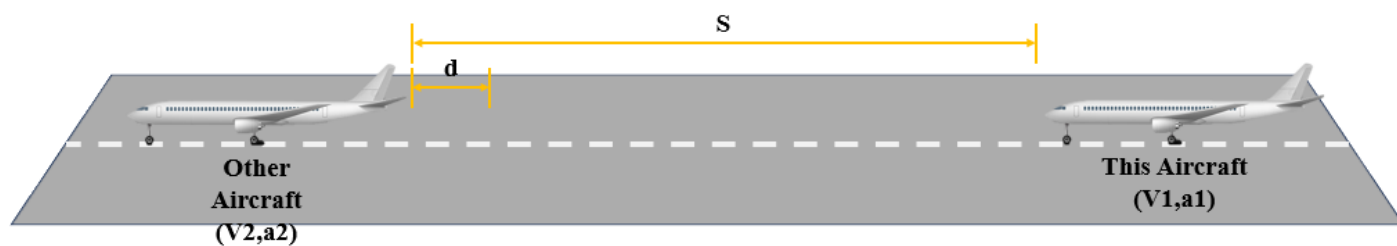
3.4 主要演算法

3.4-1 碰撞時間(Time to Collision ,TTC)

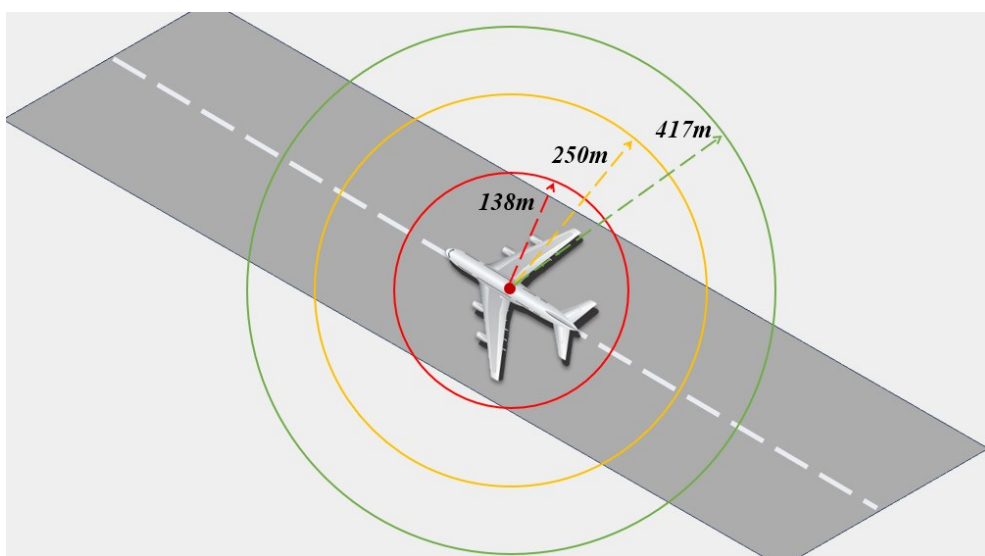
碰撞時間為兩飛機碰撞的剩餘時間，任兩台飛機當前面的飛機速度低於後面飛機時存在碰撞時間，本專題先訂定三種碰撞時間，並將相對時速定為 50km/hr(13.8m/s)，然後藉碰撞時間公式反推符合這個碰撞時間為 60、35、20 秒剩餘的相對距離，這三個距離則為本專題三種程度的警示狀態(紅、黃、綠)。

$$v_1 t + \frac{1}{2} a_1 t^2 + (s - d) = v_2 t + \frac{1}{2} a_2 t^2$$

$$T = \frac{s - d}{v_2 - v_1}$$



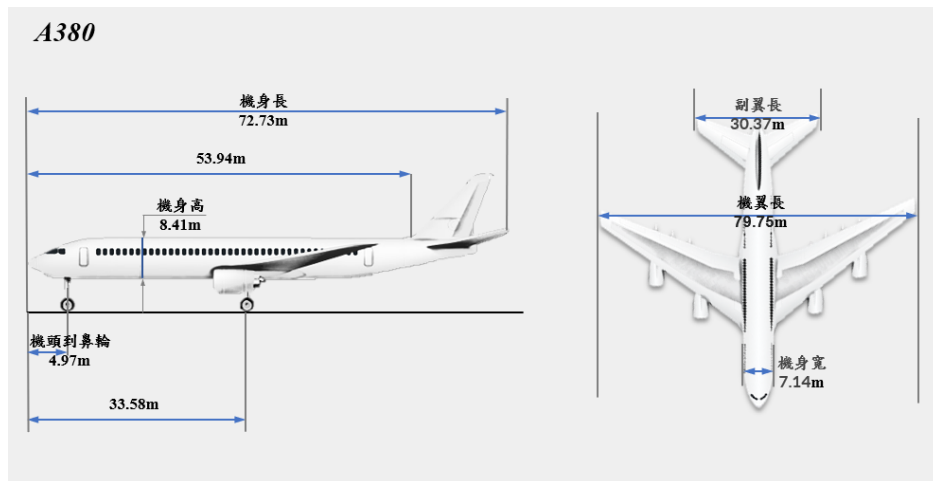
圖二十一、TTC 計算示意圖



圖二十二、TTC 範圍圖

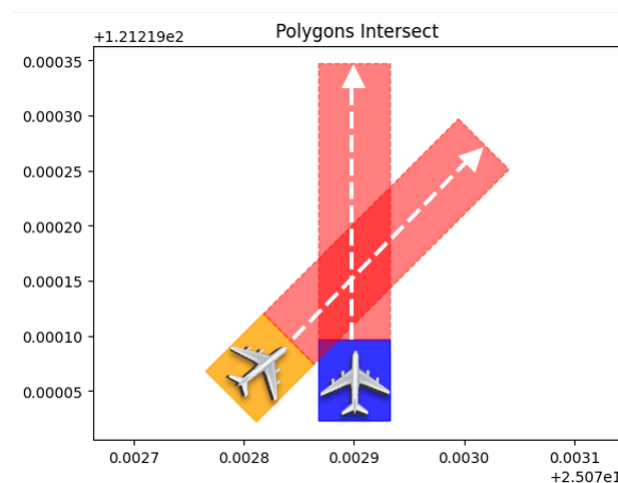
3.4-2 SAT(Separating Axis Theorem)+預測路徑

本專題同時透過分離軸定理(Separating Axis Theorem,SAT)判斷，分離軸定理是快速通用的演算法，判斷方式為如果兩個凸多邊形不相交，必然存在一條分離軸 (Separating Axis)，用於檢測兩個凸多邊形是否相交，或者用於確定一個凸多邊形是否包含另一個凸多邊形。

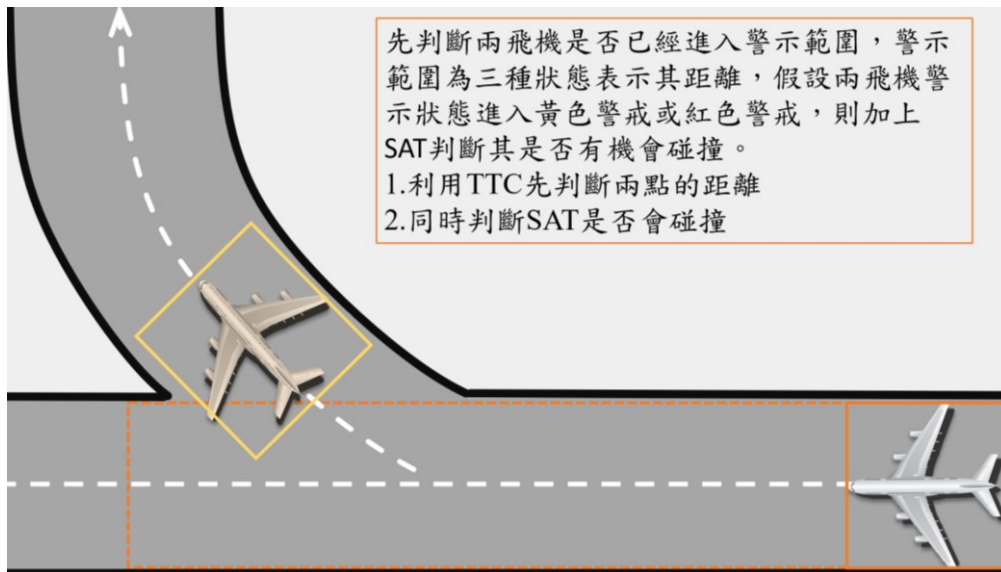


圖二十三、A380 規格圖

本專題依照航向及速度預測出飛機未來路徑，而 A380 為現役最大的客機，因此採用 A380 機身長、機翼長和預測路徑來框出 SAT 的飛機範圍儲存成長方形，因為機翼長和機身長完整包含了垂直尾翼和水平尾翼還有機頭至鼻輪，所以選擇用這兩個長度畫框，接著透過分離軸定理判斷兩台飛機未來路徑的長方形是否會相交，藉此辨別兩飛機持續以該速度航向前進是否會發生碰撞。



圖二十四、SAT 預測路徑示意圖



圖二十五、主要演算法示意圖

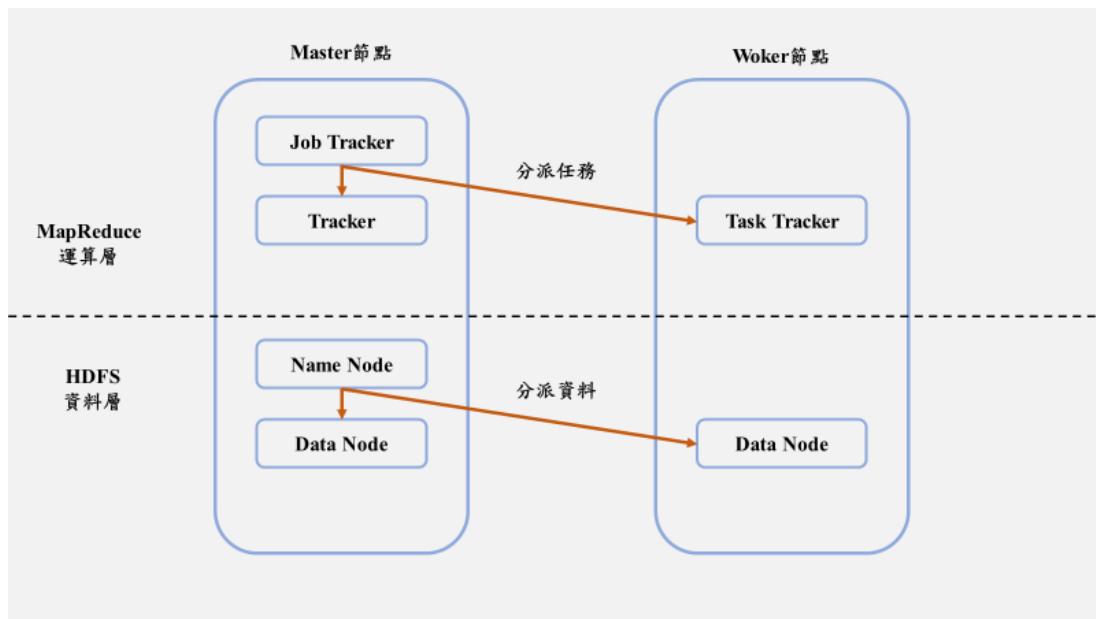
先判斷兩飛機是否已經進入警示範圍，警示範圍為三種狀態表示其距離，假設兩飛機警示狀態進入黃色警戒或紅色警戒，則加上 SAT 判斷其是否有機會碰撞。

- 1.利用 TTC 先判斷兩點的距離
- 2.同時判斷 SAT 是否會碰撞

3.5 Mapreduce

大量的資料存入 Hbase 後並不是每筆資料都是要經過我們計算的，飛機在空中本身就有一套防撞系統，所以我們透過 Mapreduce 將資料切成飛行、落地兩筆資料，在飛行中的我們會將它顯示於網頁上但不會進行預防碰撞，即將落地以及在地面上的飛機再一次 MapReduce 切分成屬於哪個機場的範圍，接著經過演算法判斷。

MapReduce 同樣使用 Hadoop 的 HDFS 系統，透過 HDFS 系統對節點分派資料，運算層則有 Job 跟 Task 的 Tracker，追蹤運算的進度，MapReduce 可以進行平行運算，在各節點執行一支可平行執行的程式，這樣可以提升處理巨量資料的速度。



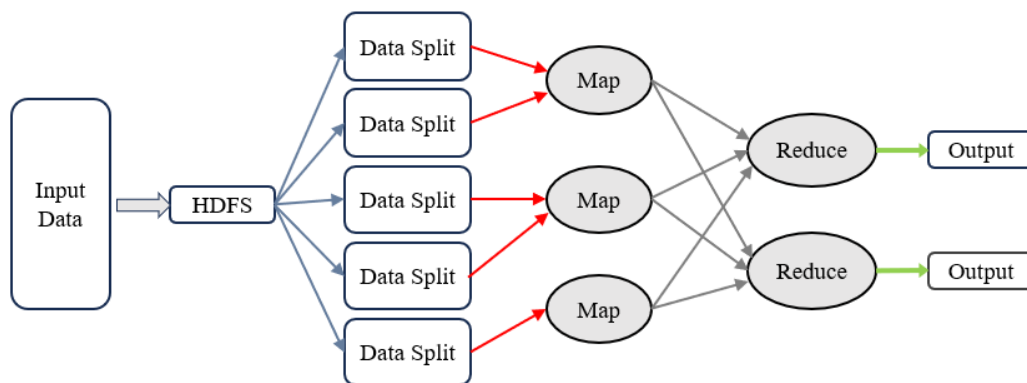
圖二十六、MapReduce 運作圖

Map:

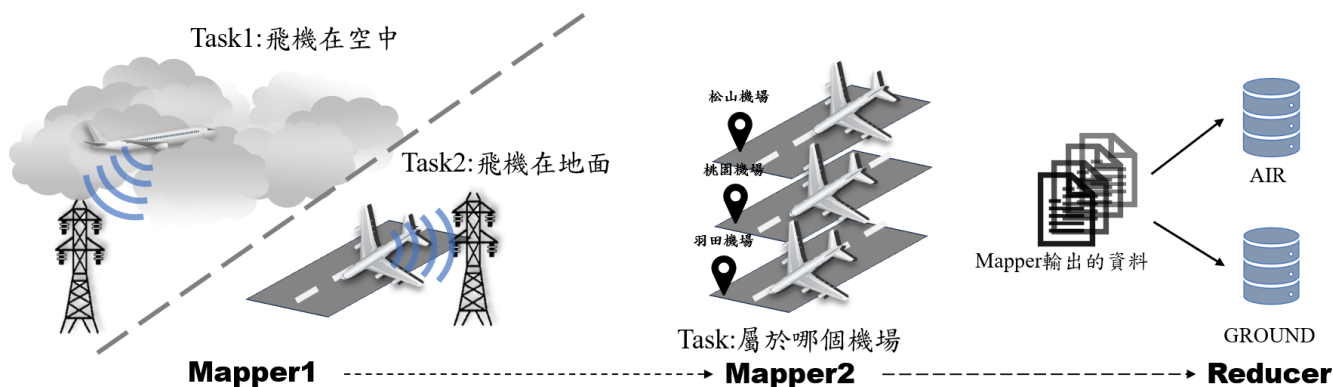
該階段會在每個節點執行相同的程式，將資料做切分並分配給每個節點做運算，加快大量資料的運算，本專題 Map 的程式為讀取 HBase 內的資料並判斷飛機是否在空中飛行，輸出固定格式的資料。

Reduce:

該階段會執行 map 階段結果的整合，讀取 Map 階段輸出的資料，做後續統整或存取，本專題 Reduce 的程式為讀取 Map 階段輸出的資料，並依照其判斷結果將資料分別存入 ground 和 air 這兩個 HBase 的 Table 內，以利後續應用。



圖二十七、MapReduce 執行過程圖



圖二十八、MapReduce 應用示意圖

3.6 網頁即時顯示

在研究中，當飛機的範圍將會互相碰到警示範圍需要一個呈現系統，而我們使用 OpenStreetMap 來做為平台，OSM 不會屏蔽附近有軍用機場的飛機場，讓我們獲得更完整的地圖資料，在網頁上不僅可以顯示三種不同的警戒，還有顯示相互有警告的飛機航號，在點擊飛機時，會有該飛機的即時資訊以及路徑。

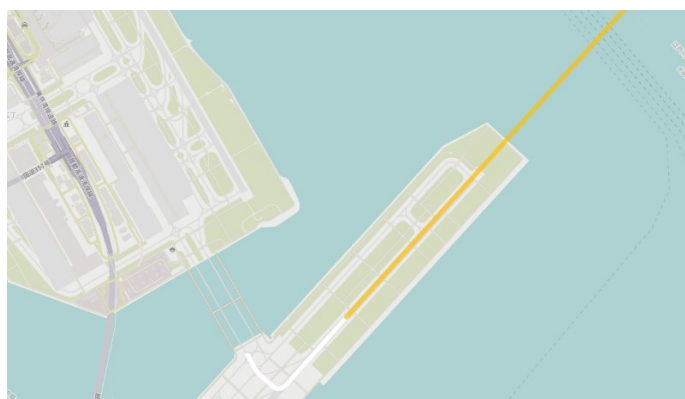
飛機顏色: 飛行中(黃)、地面上(藍)

飛機狀態: Flying(空中飛行)、Taxing(滑行)、Stop(停止)

飛機資訊: 點擊飛機後顯示航班、GPS 位置、速度、高度、航向以及排程出發、抵達時間

路徑顏色: 地面上(白)、飛行中(黃)

警示狀態: 兩飛機距離在黃圈以及紅圈範圍內會顯示圓圈，且用 SAT 偵測碰撞，如果會碰撞將再狀態下顯示警告以及即將撞上的飛機航班。



圖二十九、路徑變換顏色系統圖



圖三十、飛機高度圖示圖

機場

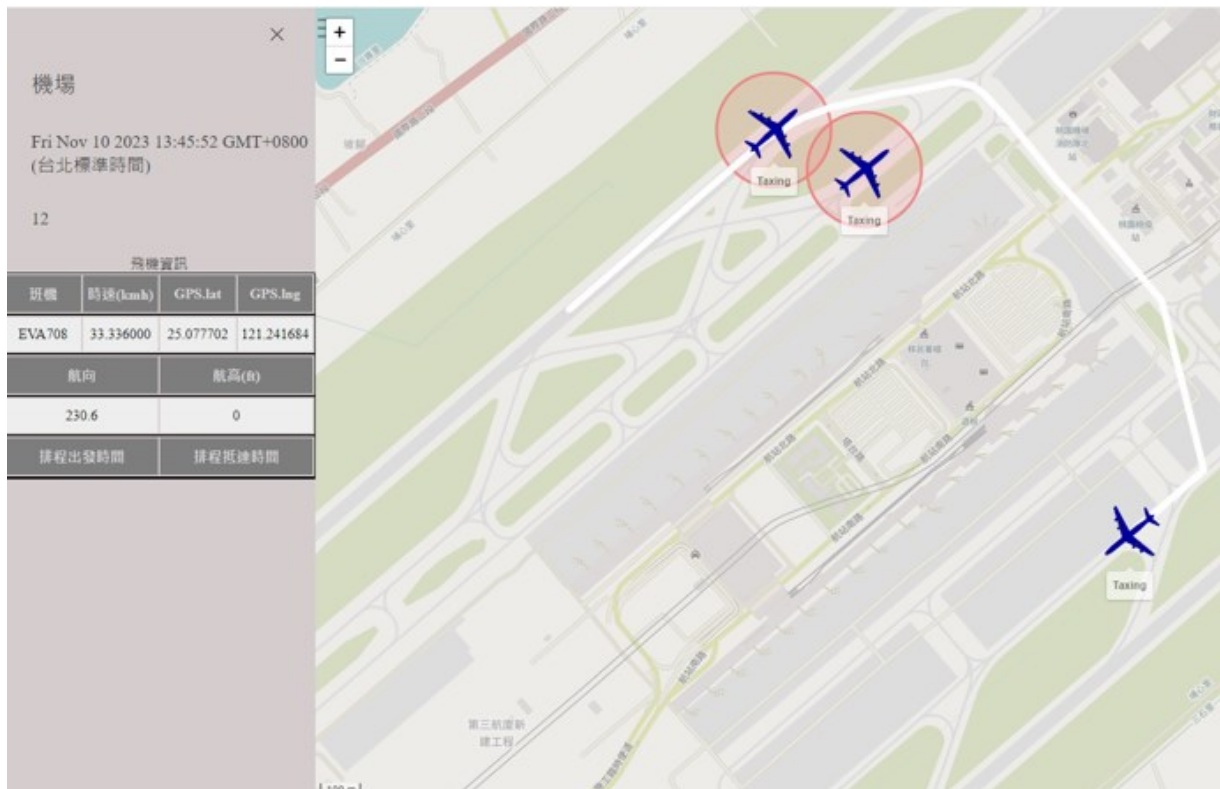
Tue Nov 07 2023 18:54:13
GMT+0800 (台北標準時間)

13

飛機資訊

班機	時速(kmh)	GPS.lat	GPS.lng
SIA606	935.260000	23.593140	120.372467
航向		航高(ft)	
18.6		41000	
排程出發時間		排程抵達時間	

圖三十一、飛機即時資訊表顯示圖



圖三十二、桃園機場系統實際測試圖

3.7 發送警示訊息系統

當兩台飛機透過我們運算判斷警告狀態成立時，會發送警告通知至 Line 的聊天室上，讓航管人員在沒有監控網頁下也能獲取警告資訊，除了發送警告，也會通知航管人員飛機建議進行煞車或閃避動作。未來也可以直接使用其他通訊系統通知塔台航管人員或者直接通知飛機駕駛者。

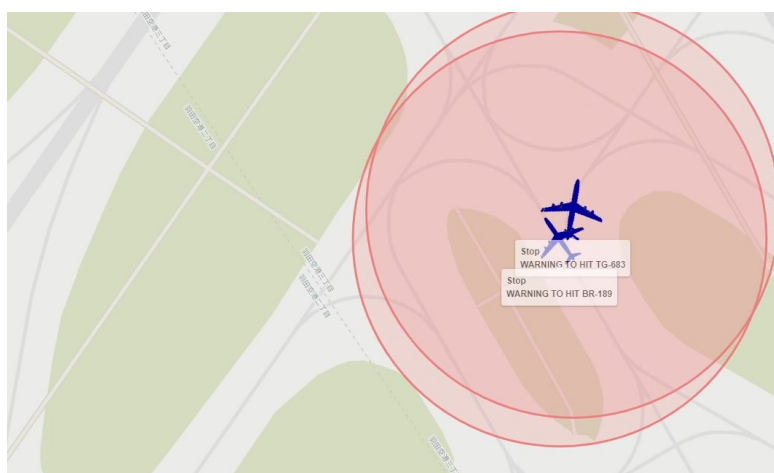


圖三十三、LINE 通知

四、系統測試

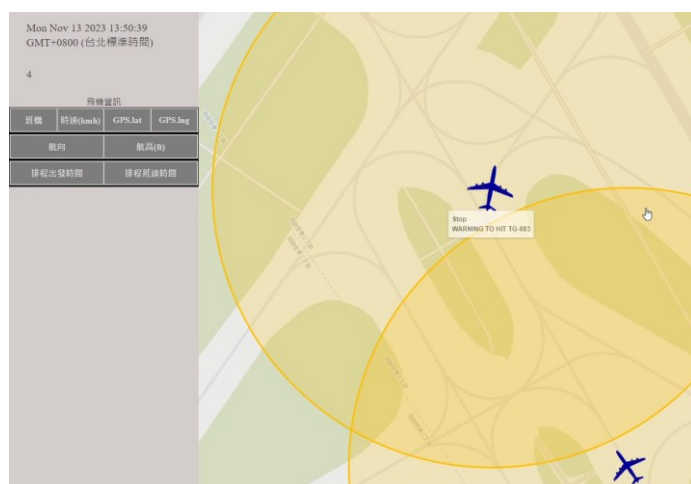
4.1 情境一

本次模擬還原了 2023 年 6 月 10 日羽田機場長榮與泰國航空發生的碰撞事故，該事故發生於羽田機場的 L14 滑行道，接著分析若利用我們的系統是否真的會做出提醒以及事故是否還會發生。



圖三十四、羽田事件還原圖

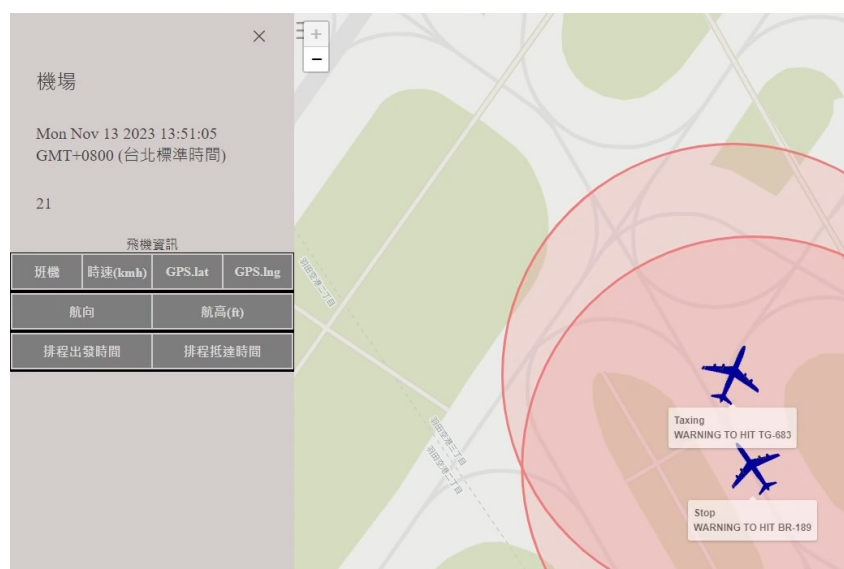
圖三十四是本專題依照所收集之資料，照當時狀況地點所還原的羽田機場碰撞事件，該事件造成多架班機延誤，機場交通大亂，事件初步判斷為長榮航空離停止線太遠，而泰國航空機長判斷失誤，誤以為能順利通過造成擦撞，若是加上本專題的系統，幫助機長判斷或是提醒管制人員，損失將大幅下降。



圖三十五、增加系統後比較圖-1



圖三十六、警告訊息發送圖



圖三十七、增加系統後比較圖-2

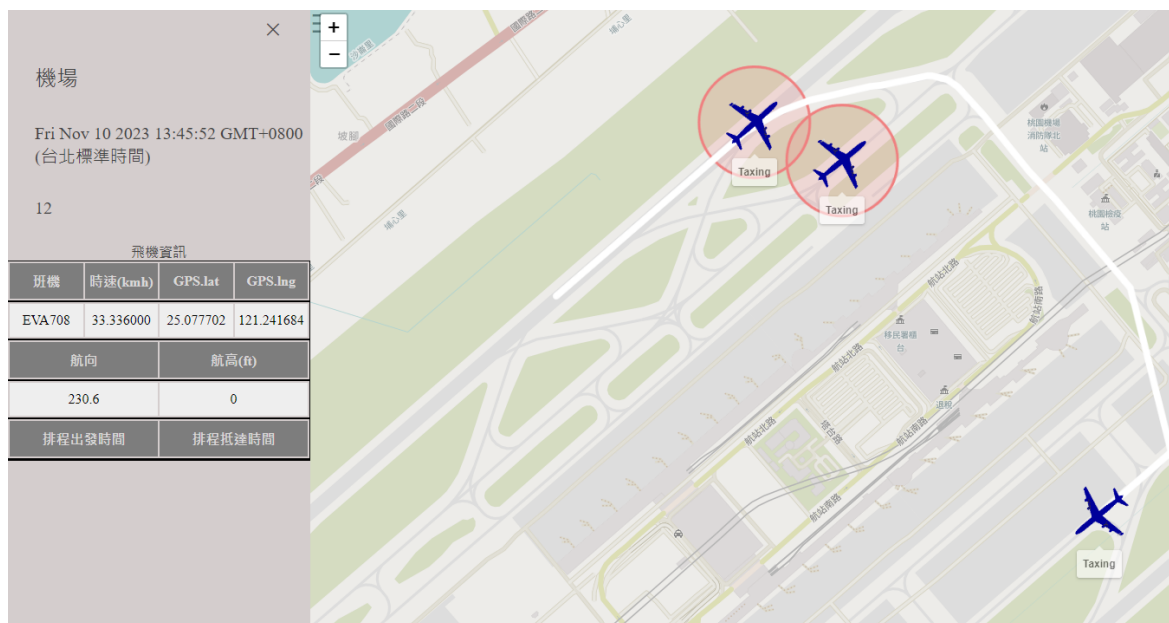
藉由圖三十五和圖三十六可以看出本系統可以即時發送警示訊息給管制人員，使其能夠指揮後方的班機執行停下的動作避免發生碰撞，而圖三十六和圖三十七可以得知飛機收到警示訊息後確實有停下沒有如圖三十四直接撞上，且管制人員和機長可以反應的時間大約在 40 至 50 秒。

	有系統輔助	無系統輔助(人工)
反應時間	40~50s 提前通知有狀況	不一定，有可能沒注意
警示提醒	有	無
人力成本	低，可以偵測整座機場	高，每人僅能注意少量飛機
事故發生機率	低，已提前警示	較高，沒有注意或誤判就會有事故

表四、有無系統輔助比較表

4.2 機場實際測試

我們認為本系統僅透過模擬還原事件可能還無法確認其實際作用時的穩定性以及訊號延遲等等狀況，因此決定前往桃園國際機場實際測試本系統。

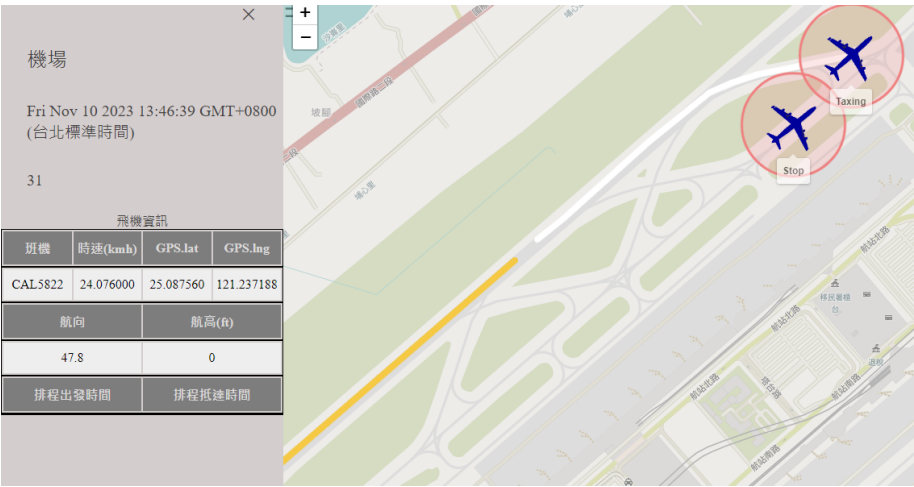


圖三十八、桃園機場實測圖



圖三十九、機場飛機實拍圖

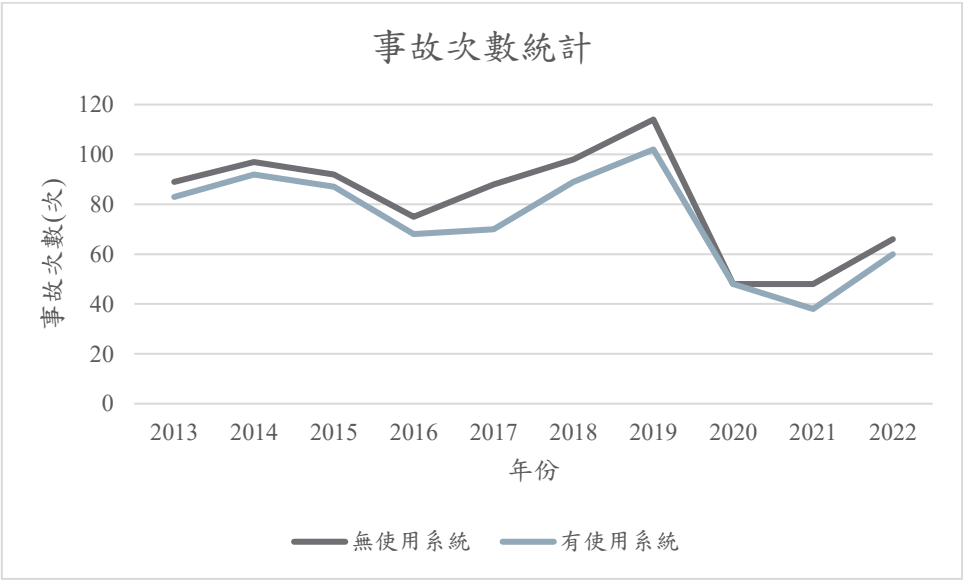
當天也有遇到兩架飛機非常靠近，系統也有改變警示狀態，所以證明系統是可以正常運作，且沒有延遲問題。



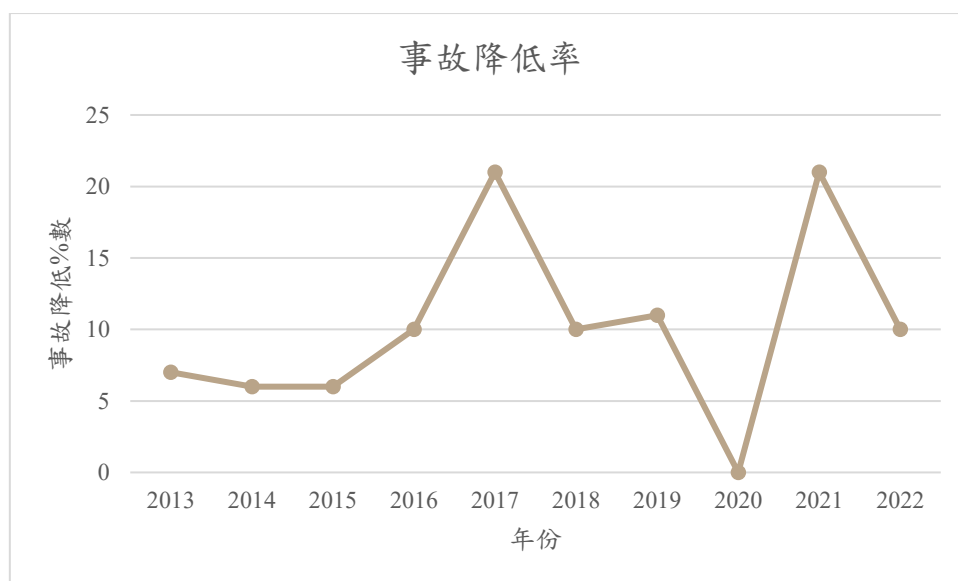
圖四十、系統狀態改變圖

五、 結論

飛機在地面上發生碰撞的事件其實持續都有在世界各地發生，只是嚴重程度不一，但是飛機其相對其他交通工具精密許多，任何一點小碰撞或擦撞都需要先停飛檢修，這樣就會延誤旅客時間若是嚴重的事件還需封閉跑道影響更多班機，因此要降低飛機在地面發生碰撞，根據 ICAO 的統計每年大約有 7~10% 以上的事故是在飛機 Taxing 時發生，我們的系統能有效減少 Taxing 時的事故發生的話數據會如下表。



表五、事故次數統計表



表六、事故降低率

透過圖表可以得知減少 Taxing 時的碰撞可以有效降低總事故次數，平均將每年的事故降低 10%，所以飛機在地面滑行的防碰撞系統是非常重要且需要的，專題研究如何透過軟體輔助，不增加飛機上的感測器為前提開發系統，通過接收 ADS-B 訊號得知飛機資訊，再經由程式判斷威脅狀態、碰撞風險等，最後提前發送通知且顯示於網頁，藉此提醒機長和管制人員做出反應和注意狀況，達到減少碰撞的目的。

專題經過模擬以及實際測試均能正常運作，而且也會依照演算法偵測來即時改變警示狀態，警示訊息的發送也沒有延遲，大約會在偵測碰撞發生的 40~50 秒前發送，網頁的飛機資訊監測功能也能正常運作。

我們希望專題開發的系統能夠幫助到機場的運作，尤其是吞吐量大的大型機場，這些機場非常繁忙，人工指揮與調度下容易因人疲勞出現失誤，此時這個系統就能派上用場，幫助這些管制人員監控飛機，降低事故的發生率，同時也可免除事故後續的成本。

六、 参考文献

- [1] A. Khedkar and V. Kumar, “Self synchronization based air traffic control and collision avoidance system,” *2011 IEEE/AIAA 30th Digital Avionics Systems Conference*, Seattle, WA, USA, 2011.
- [2] Y. Li, L. Zhang and Y. Song, “A vehicular collision warning algorithm based on the time-to-collision estimation under connected environment,” *2016 14th International Conference on Control, Automation, Robotics and Vision (ICARCV)*, Phuket, Thailand, 2016.
- [3] I. Lee, K. K. Lee, O. Sim, K. S. Woo, C. Buyoun and J. -H. Oh, “Collision detection system for the practical use of the humanoid robot,” *2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*, Seoul, Korea (South), 2015.
- [4] D. Yun, Y. Yong, X. Kaijun and X. Zhaoyu, “Progress of ADS-B IN technology in civil aviation applications,” *2022 7th International Conference on Communication, Image and Signal Processing (CCISP)*, Chengdu, China, 2022.
- [5] M. Shravan, R. Rakshit, P. Sanjana, B. K. Priya and N. Kumar, “RTL SDR ADS-B Data Analysis for Predicting Airports and ATS Routes,” *2020 International Conference for Emerging Technology (INCET)*, Belgaum, India, 2020.